**Paper Number 392**

# SVM Binary Classifier Ensembles for Image Classification

**Kingshy Goh, Edward Chang and Kwang-Ting Cheng**
Electrical & Computer Engineering
University of California, Santa Barbara
echang@ece.ucsb.edu

**Abstract**

We study how the SVM-based binary classifiers can be effectively combined to tackle the multi-class image classification problem. We study several ensemble schemes, including OPC (one per class), PWC (pairwise coupling), and ECOC (error-correction output coding), that aim to achieve good error correction capability through redundancy. To enhance these ensemble schemes' accuracy, we propose methods that on the one hand boost the margins (i.e., confidence) of the SVM-based binary classifiers, and, on the other hand, remove the noise of irrelevant classifiers from class prediction. From empirical study we show that our margin boosting and noise reduction methods lead to higher classification accuracy than ensemble schemes that are solely designed for maximum error correction capability.

## 1 Introduction

Being able to classify images automatically and accurately is important for both effective image organization and retrieval. Achieving high image classification accuracy, however, is quite challenging. This is partly because images are diversified and each can be characterized by a large set of different features, and partly because semantically related images may not reside continuously (and thus, may not be linearly separable) in the feature space. In this paper, we focus our study on image classification methods. (We discuss image characterization in [6] and feature selection in [5, 13, 18].)

Although the regression approach considers the discrimination problem of $K$ classes simultaneously, it can be computationally intensive ($O(K^3)$) and may impose a universal assumption on all class boundaries (see Friedman [8]). Of late, many studies have proposed ensemble schemes, which use binary (two-class) classification algorithms to solve $K$-class

classification problems. Decomposing a $K$-class classification problem into a number of binary classification problems allows an ensemble scheme to model binary class boundaries with much greater flexibility at a lower computational cost (in the order of $O(K^2)$) than the regression approach. Three representative ensemble schemes are *one per class*, *pairwise coupling*, and *error-correcting output coding*.

**1**. One per class (OPC). This is also known as "one against others." OPC trains $K$ binary classifiers, each of which separates one class from the other $(K-1)$ classes. Given a point $X$ to classify, the binary classifier with the largest output determines the class of $X$.

**2**. Pairwise coupling (PWC). PWC constructs $K(K-1)/2$ pairwise binary classifiers. The classifying decision is made by aggregating the outputs of the pairwise classifiers.

**3**. Error-correcting output coding (ECOC). ECOC was first proposed by Dietterich and Bakiri [7] to reduce classification error by exploiting the redundancy of the coding scheme. ECOC employs a set of binary classifiers assigned with codewords such that the Hamming distance between each pair is far enough apart to enable good error correction.

In this paper, we propose margin boosting and error reduction methods to improve class prediction accuracy of OPC, PWC, and ECOC. Our study follows three steps:

**1**. Choosing a binary classifier. We employ Support Vector Machines (SVMs) as the binary classifier. We evaluate SVM kernels and parameter settings for the image classification task.

**2**. Evaluating existing ensemble schemes. We evaluate OPC, PWC, and ECOC on a diversified image dataset. While evaluating these traditional schemes, we identify two sources of misclassification:

- *Strong classifiers with weak voice.* Some accurate binary classifiers have weak influence in making a class prediction.
- *Irrelevant classifiers.* Some irrelevant classifiers have strong influence in making a class prediction.

**3**. Proposing remedies. To remedy the above observed problems, we propose two methods:

- *Margin boosting.* We carefully boost the confidence of a binary classifier that is relevant to the prediction of the final class outcome. When this is done, a prediction is less susceptible to prediction variance and thus more accurate. (Figure 4 in Section 4 explains the effect of margin boosting.)
- *Error reduction.* We employ an error reduction scheme that reduces the effect from the noise of irrelevant classifiers.

Through empirical study and analysis, we show that our remedies can improve class prediction accuracy for OPC, PWC, and ECOC. We have also discovered that although ECOC is designed for error-correction, OPC consistently outperforms ECOC in our experiments.

The rest of this paper is organized into five sections. Section 2 briefly introduces SVMs. Section 3 introduces the OPC, PWC, and ECOC ensemble models. Section 4 explains the problems associated with these three models and our margin-boosting and error-reduction remedies. In Section 5, we report experimental results showing that our remedies can improve class prediction accuracy for all ensemble schemes. Finally, we offer our observations and conclusions in Section 6.

### 1.1 Related Work

Ensemble methods can be divided into two categories based on their design goals. The goal of the first category is to reduce prediction variance caused by training data selection. Some well-known methods in this category include bagging [2], arcing [3], and boosting [17]. This study does not focus on variance reduction. (Our work does not reduce variance directly, but can reduce the effect of variance on prediction accuracy.)

The goal of the second ensemble category is to reduce prediction error using decomposition and reconstruction methods with good error-correction capability [1, 7, 19]. In this study, we show that such an ensemble scheme may not perform better than a scheme like OPC that lacks good error-correction capability. We argue that in designing a robust ensemble scheme, error recovery is secondary to error reduction.

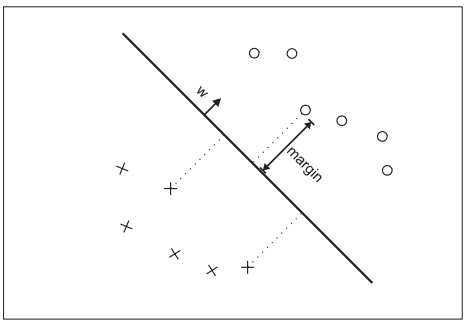



Figure 1: A Simple Linear Support Vector Machine.

## 2 Support Vector Machines

Support vector machines [20, 21, 12] are a core machine learning technology. They have strong theoretical foundations and excellent empirical successes. We shall consider SVMs in the binary classification setting. We are given training data $\{\mathbf{x_1} \ldots \mathbf{x_n}\}$ that are vectors in some space $\mathcal{X} \subseteq \mathbb{R}^{\lceil}$. We are also given their labels $\{y_1 \ldots y_n\}$ where $y_i \in \{-1, 1\}$. In their simplest form, SVMs are hyperplanes that separate the training data by a maximal margin (see Figure 1) . All vectors lying on one side of the hyperplane are labeled as $-1$, and all vectors lying on the other side are labeled as 1. The training instances that lie closest to the hyperplane are called *support vectors*. More generally, SVMs allow us to project the original training data in space $\mathcal{X}$ to a higher dimensional feature space $\mathcal{F}$ via a Mercer kernel operator $K$. In other words, we consider the set of classifiers of the form: $f(\mathbf{x}) = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{K}(\mathbf{x_i}, \mathbf{x})$. When $f(\mathbf{x}) \geq \mathbf{0}$ we classify $\mathbf{x}$ as $+1$; otherwise we classify $\mathbf{x}$ as $-1$.

When $K$ satisfies Mercer's condition [4] we can write: $K(\mathbf{u}, \mathbf{v}) = \mathbf{\Phi}(\mathbf{u}) \cdot \mathbf{\Phi}(\mathbf{v})$ where $\Phi : \mathcal{X} \to \mathcal{F}$ and "$\cdot$" denotes an inner product. We can then rewrite $f$ as:

$$f(\mathbf{x}) = \mathbf{w} \cdot \mathbf{\Phi}(\mathbf{x}), \text{ where } \mathbf{w} = \sum_{\mathbf{i=1}}^{\mathbf{n}} \alpha_\mathbf{i} \mathbf{\Phi}(\mathbf{x_i}). \tag{1}$$

Thus, by using $K$ we are implicitly projecting the training data into a different feature space $\mathcal{F}$, often one of higher dimension. The SVM then computes the $\alpha_i$s that correspond to the maximal margin hyperplane in $\mathcal{F}$. By choosing different kernel functions we can

implicitly project the training data from $\mathcal{X}$ into spaces $\mathcal{F}$. The hyperplanes in $\mathcal{F}$ correspond to more complex decision boundaries in the original space $\mathcal{X}$.

Two commonly used kernels are the polynomial kernel and radial basis function (RBF) kernel. The polynomial kernel $K(\mathbf{u}, \mathbf{v}) = (\mathbf{u} \cdot \mathbf{v} + \mathbf{1})^{\mathbf{p}}$ induces polynomial boundaries of degree $p$ in the original space $\mathcal{X}$. The Gaussian RBF kernel $K(\mathbf{u}, \mathbf{v}) = (\mathbf{e}^{-\gamma(\mathbf{u}-\mathbf{v})\cdot(\mathbf{u}-\mathbf{v})})$ induces boundaries by placing weighted Gaussians upon key training instances. Another common RBF kernel is the Laplacian kernel given by $K(\mathbf{u}, \mathbf{v}) = (\mathbf{e}^{-\gamma \sum_i |\mathbf{u_i}-\mathbf{v_i}|})$. It can be interpreted as representing the $L_1$ distance between points in $\mathcal{X}$.

# 3 Traditional Ensemble Schemes

A classification problem involves a function $F : \Omega \rightarrow 1, \ldots, K$, which defines a $K$-partition of the input space $\Omega$ into sets $F^{-1}(k)$ called *classes* denoted by $\omega_k$, where $1 \leq k \leq K$. For a two-class problem, the hyperplane $\mathbf{w}$ of SVMs is the partition and $sgn(f(\mathbf{x}))$ is equivalent to $F^{-1}(k)$ .

Although SVMs are mainly for two-class problems, they can be adapted to multi-class problems. Three representatives methods are: OPC, PWC, and ECOC.

## 3.1 One per Class (OPC)

OPC is also known as "one against others." OPC is simple and gives reasonable performance. We train $K$ SVM classifiers, each of which separates one class from the other $(K-1)$ classes. For each point, we have $K$ SVM decision outputs $f_k(\mathbf{x})$, $1 \leq k \leq K$. The class of a point is given as $\arg\max_k f_k(\mathbf{x})$.

## 3.2 Pairwise Coupling (PWC)

PWC trains $\frac{1}{2}K(K-1)$ binary SVM classifiers. To classify a data object, PWC combines the scores of these $\frac{1}{2}K(K-1)$ classifiers. In the remainder of this section, we depict how the scores can be aggregated and a classification decision can be made.

Each of the $\frac{1}{2}K(K-1)$ binary SVM classifiers provides a partial decision for classifying

a point. The study of [14] shows that combining these decision outputs differently may yield different class decisions.

A binary classifier decides whether a point $x$ belongs to class $\omega_i$ or $\omega_j$. The probability that $x$ belongs to class $\omega_i$, given that $x$ is in either class $\omega_i$ or $\omega_j$, can be written as

$$\tilde{p}_{ij} = P(\mathbf{x} \, \epsilon \, \omega_\mathbf{i} \mid \mathbf{x}, \;\; \mathbf{x} \, \epsilon \, \omega_\mathbf{i} \, \cup \, \omega_\mathbf{j}). \tag{2}$$

With $\tilde{p}_{ij}$, we can calculate the estimate $\tilde{p}_i$ of the a posteriori probability

$$\tilde{p}_i = P(\mathbf{x} \, \epsilon \, \omega_\mathbf{i} \mid \mathbf{x}) \tag{3}$$

by using a matrix of $\tilde{p}_{ij}$ and $\tilde{p}_{ji} = 1 - \tilde{p}_{ij}$ to compute $\tilde{p}_i$ as follows:

$$\tilde{p}_i = \frac{2}{K(K-1)} \sum_{j \neq i} \tilde{p}_{ij}, \tag{4}$$

and the classification is given by

$$\arg \max_{1 \leq i \leq k} \tilde{p}_i. \tag{5}$$

The SVM decision output $f_{ij}(\mathbf{x})$ is not a probability value. However, we can first normalize it (details to be presented in Section 4.1) such that $f_{ij}(\mathbf{x}) \, \epsilon \, \{\mathbf{-1}, \mathbf{+1}\}$. We then map the normalized output to the probability $\tilde{p}_{ij}$ according to the following intuitive transformation:

$$\tilde{p}_{ij} = \frac{1}{2} f_{ij}(\mathbf{x}) + \mathbf{0.5}. \tag{6}$$

We can generalize $\tilde{p}_{ij}$ in Equation 4 to a class decision function $\sigma(\tilde{p}_{ij})$. We evaluate four forms of $\sigma(\tilde{p}_{ij})$) that are proposed in [14] as follows:

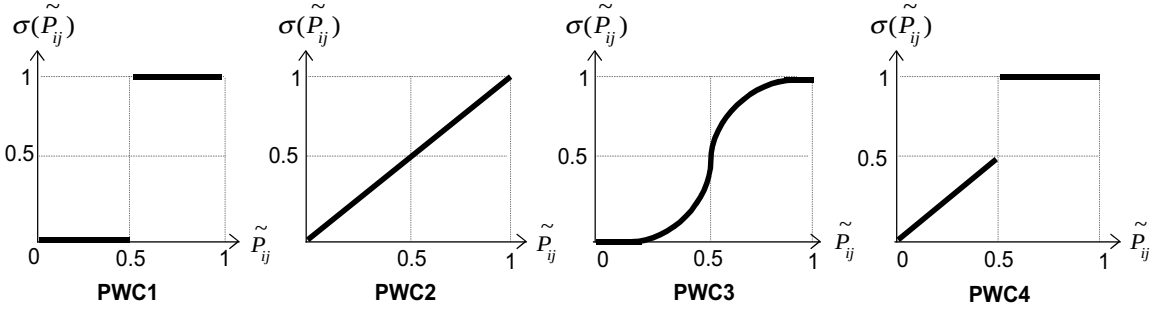



Figure 2: Four Representative Class Decision Functions.

$$
\begin{aligned}
\sigma(\tilde{p}_{ij}) &= \begin{cases} 1 & \text{if } \tilde{p}_{ij} \geq 0.5 \\ 0 & \text{otherwise} \end{cases} && \text{(PWC1)} \\
\sigma(\tilde{p}_{ij}) &= \tilde{p}_{ij} && \text{(PWC2)} \\
\sigma(\tilde{p}_{ij}) &= \frac{1}{1+e^{-12(\tilde{p}_{ij}-0.5)}} && \text{(PWC3)} \\
\sigma(\tilde{p}_{ij}) &= \begin{cases} 1 & \text{if } \tilde{p}_{ij} \geq 0.5 \\ x & \text{otherwise} \end{cases} && \text{(PWC4)} \qquad (7)
\end{aligned}
$$

Figure 2 shows the plots of the four $\sigma(\tilde{p}_{ij})$. Scheme PWC1 is a threshold combination. PWC2 is a linear combination of the SVM decision outputs. PWC3 is a compromise between PWC1 and PWC2. PWC4 assigns greater importance to positive SVM decision outputs than to negative ones close to the SVM margin. Finally, we can rewrite Equation 4 as

$$\tilde{p}_i = \frac{2}{K(K-1)} \sum_{j \neq i} \sigma(\tilde{p}_{ij}) \qquad (8)$$

### 3.3 Error-Correcting Output Coding (ECOC)

Error-correcting output coding (ECOC) proposed by Dietterich and Bakiri [7] is another method for combining classifiers produced by a binary learning algorithm to solve. Each class is given a unique binary string of length $L$ called codewords. $L$ binary functions are learned to produce $L$ binary classifiers. Table 1 shows a set of 15-bit codewords assigned to five classes, with the columns $f_0, ..., f_{14}$ representing the classifiers to be learned. All columns are distinct and each classifier learned is a disjunction of the classes. For example, $f_1(x) = 1$ if $x$ is in class 1 or 5.

| Class | $f_0$ | $f_1$ | $f_2$ | $f_3$ | $f_4$ | $f_5$ | $f_6$ | $f_7$ | $f_8$ | $f_9$ | $f_{10}$ | $f_{11}$ | $f_{12}$ | $f_{13}$ | $f_{14}$ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 2 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| 3 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| 4 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 1 |
| 5 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 |

Table 1: Error-correcting output codes for five-class problem with minimum Hamming distance of eight.

ECOC is robust to changes in the size of the training dataset and to the assignment of codewords to classes. ECOC claims to have good error-correction capability, which is measured by the minimum Hamming distance $d$ between any pair of codewords. Each codeword is able to correct at least $\lfloor \frac{d-1}{2} \rfloor$ single-bit errors. For example, the set of codewords in Table 1 has a Hamming distance of eight, allowing it to correct up to three single-bit errors. (The error-correction capability of OPC is zero.) This error-correction capability is dependent on the fact that the errors committed by each bit of the output are uncorrelated.

In order to have good error-correction capability, the codewords should possess the following properties:

- **Row separation**. Each codeword should be well separated from each other in terms of Hamming distance.
- **Column separation**. Each bit in a column $f_i$ should be uncorrelated with another column $f_j$, where $j \neq i$. There should be a large Hamming distance between different column, and also between a column and the complement of other columns. This ensures that the columns are neither identical nor complementary.

Given $K \leq 7$ classes, an exhaustive method can be used to select the optimal codewords. For larger $K$, a randomized hill-climbing algorithm is used. First, $K$ random strings of codewords with length $L$ are generated. Next, the algorithm attempts to improve the minimum Hamming distance by searching for pairs of rows closest to each other, and columns that are either too close or too far apart. The four codeword bits that the two rows and columns intersect are located and changed to improve their separation. In our experiments, we will

compare the performance of the initial randomly generated codewords with the final set produced by the algorithm.

To classify a test example $X$, the $L$ SVM outputs $f_0, ...f_L$ are combined to generate a codeword $C = (c_1, c_2, ....c_L)$ as follows:

$$c_i = \begin{cases} 1 & \text{if } f_i \geq 0 \\ 0 & \text{otherwise} \end{cases} \tag{9}$$

The generated codeword $C$ is compared to the pre-assigned codewords for each class; the closest class in terms of Hamming distance between the codewords gives the classification of $X$.

# 4 Margin Boosting and Error Reduction

In this section, we propose two remedies, *margin boosting* and *error reduction*, for improving class prediction accuracy of OPC, PWC, and ECOC. We first discuss the major shortcomings of the existing ensemble schemes. We then describe how we remedy these shortcomings.

The SVM-based ensemble schemes that we have described so far suffer from three shortcomings:

**1**. *SVM margin variance*. In order to use SVMs for a multi-class problem, the $K$-class classification problem is divided into several two-class sub-problems. The results from the sub-problems are then combined using the OPC, PWC, or ECOC schemes discussed in Section 3. An assumption is made that all the outputs from the sub-problems fall in the same range. However, the SVM output scale created to accommodate the support vectors of each sub-problem may be different. Figure 3 shows the SVM margins produced by the training data of OPC and ECOC for two image classes, classes 8 and 13. (Detailed experimental results are shown in Section 5.) Note that the margin scales of different image classes are different. For class 8, the margin scale is between 1 and 2.1; for class 13, between 1 and 2.4. The ECOC scheme produces margins ranging from 1 to 3 for class

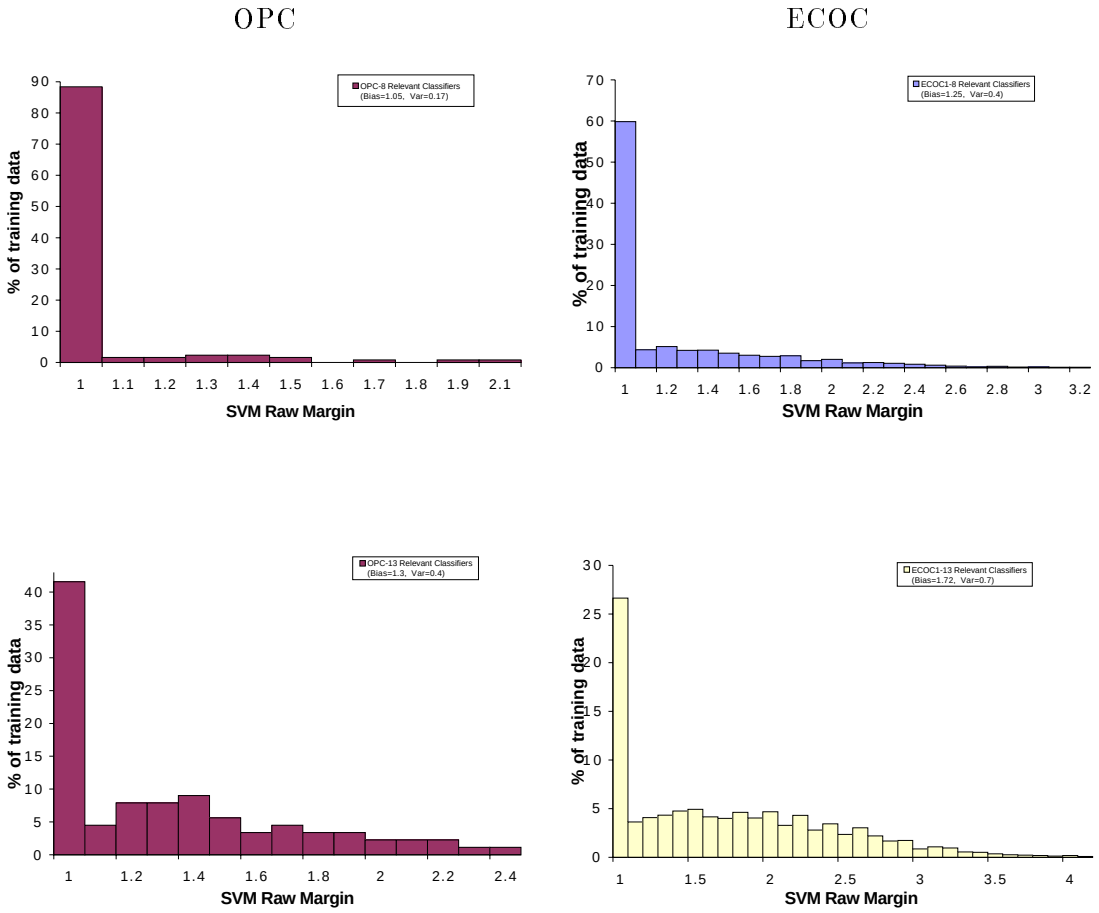
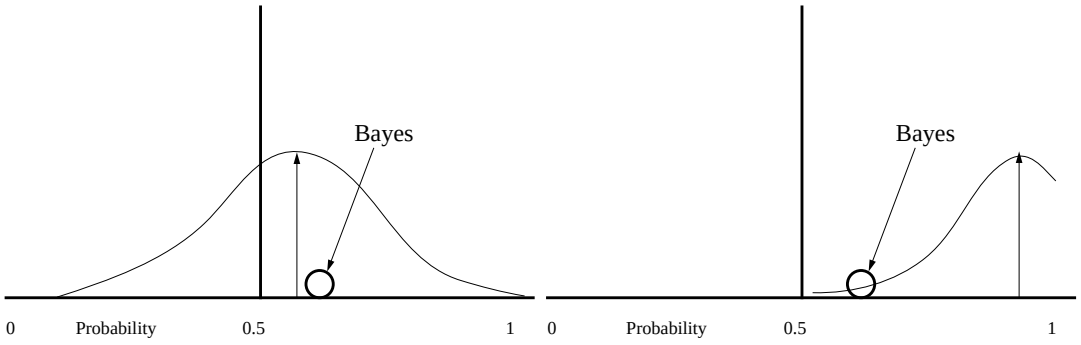



Figure 3: Example SVM Raw Margins of Training Data

8 and from 1 to 4 for class 13. Differences in scales of various classes can make class prediction difficult.

**2**. *Weak classifiers.* A weak classifier that has little confidence in its prediction is highly susceptible to variance caused by training data leading to wrong predictions. Similarly, using different decision functions on a weak classifier leads to ambiguous prediction. To illustrate, Figure 4 shows two classifiers. The $x$-axis in the figure shows the probability that point $X$ is predicted to be in class $\omega_i$. As long as the probability is greater than 0.5, point $X$ is said to be in class $\omega_i$. The circle in each subfigure indicates the prediction of Bayes. In this example, the predictions of both classifiers are consistent with Bayes and hence




(a) Weak Classifier (b) Strong Classifier

Figure 4: Variance vs. Margins

accurate. However, the weak prediction in Figure 4(a) has a high probability of being swayed by variance to the other side of 0.5 (resulting in a wrong prediction). Conversely, the confident prediction of the classifier in Figure 4(b) is unlikely to be affected by variance. Let us closely examine the effect of decision functions presented in Figure 2 on a weak classifier. Suppose the $\tilde{p}_{ij}$ matrix of point $X$ is the following:

$$\begin{pmatrix} - & 0.6 & 0.55 \\ 0.4 & - & 0.9 \\ 0.45 & 0.1 & - \end{pmatrix} \tag{10}$$

If we use decision function PWC1 or PWC4, Equations 7 and 8 will classify $X$ as class 1. If we use decision function PWC2 or PWC3, $X$ is classified as class 2. Intuitively, the closer $\tilde{p}_{ij}$ is to either zero or one (i.e., the higher the classifier's confidence in its decision), the higher the chance that all functions would classify $X$ as the same class. In other words, increasing a classifier's confidence can reduce decision variance.

**3**. *Irrelevant classifiers.* Another observation made from the $\tilde{p}_{ij}$ matrix is that there could be irrelevant classifiers causing an ensemble scheme to make a wrong prediction. Suppose point $X$ belongs to class 1. The matrix shows that both $\tilde{p}_{12}$ and $\tilde{p}_{13}$ from the binary classifiers in the first row classify $X$ correctly. $\tilde{p}_{23}$ from the classifier in the second row, although irrelevant to the decision making process, predicts $X$ as class 2 with high confidence ($\tilde{p}_{23} = 0.9$). This irrelevant decision can cause PWC2 and PWC3 to make a wrong prediction.

Figure 5 shows why an irrelevant classifier may display high confidence and cause misclas-

sification. The shaded test point sits on two class boundaries: class $1-2$ and class $1-3$. Both class $1-2$ and class $1-3$ classifiers weakly predict that the point belongs to class 1. However, the class $2-3$ classifier in this example can confidently say that the point is in class 2. This is a classification variance problem to which the PWC ensemble scheme is particularly vulnerable. We will show how this problem can be remedied in Section 4.2.

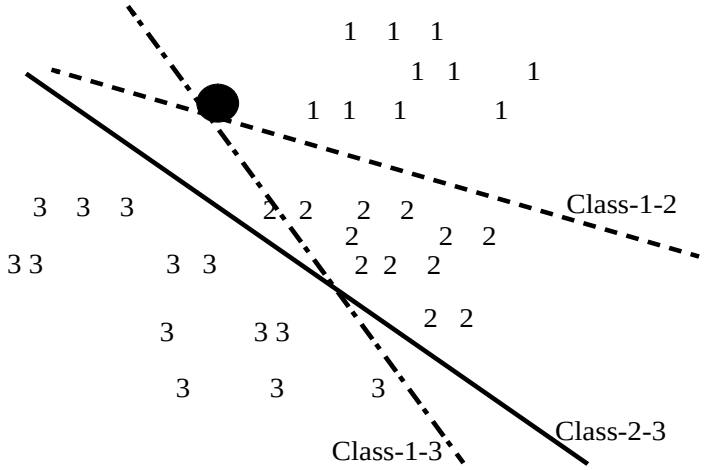



Figure 5: A Noisy Irrelevant Classifier Example.

In the remainder of this section, we describe two remedies for the above shortcomings: *margin boosting* and *error reduction*.

## 4.1 Margin Boosting

To address the first two shortcomings (SVM margin variance and weak classifiers), Platt proposes sigmoid fitting [15] to map the SVM outputs into posterior probabilities. Because Platt's method may still suffer from the *weak-classifier* problem, we propose our margin boosting method. We discuss these two methods in the following:

### 4.1.1 Platt's Sigmoid Fitting

Platt suggests that one possible way of fitting probabilities is to use Gaussians to form class-conditional probabilities $p(f|y = \pm 1)$. Bayes' rule can then be used to compute the posterior probability as:

$$P(y=1|f) = \frac{p(f|y=1)P(y=1)}{\sum_{i=-1,1} p(f|y=i)P(y=i)}, \tag{11}$$

where $P(y=i)$ are prior probabilities calculated from the training set.

We can also use a parametric model to fit the posterior $P(y=1|f)$ directly without having to estimate the class-conditional densities $p(f|y)$. According to empirical data, the class-conditional densities between SVM margins are exponential. Bayes' rule from Equation 11 on two exponentials implies that the form of the sigmoid function is

$$P(y=1|f) = \frac{1}{1+exp(Af+B)}. \tag{12}$$

This model assumes that the SVM outputs are proportional to the log odds of a positive example. The parameters $A$ and $B$ of Equation 12 are fitted using maximum likelihood estimation from a training set. $A$ and $B$ are obtained by minimizing the negative log likelihood of the sigmoid training data using a model-trust minimization algorithm. The target probabilities are derived from Bayes' rule. Given $N_+$ positive examples, the target probability for positive examples is

$$t_+ = \frac{N_+ + 1}{N_+ + 2}. \tag{13}$$

and for $N_-$ negative examples, the target probability is

$$t_- = \frac{1}{N_- + 2}. \tag{14}$$

To avoid biased sigmoid fits, we use a training set different from the data used for SVM training. A set of $A$ and $B$ is trained for each classifier of each scheme.

#### 4.1.2 Margin Boosting

In addition to the trained sigmoid function, we experimented with a fixed sigmoid function for all classifiers in all schemes using $A = -3$ and $B = -0.2$. Such a sigmoid will polarize

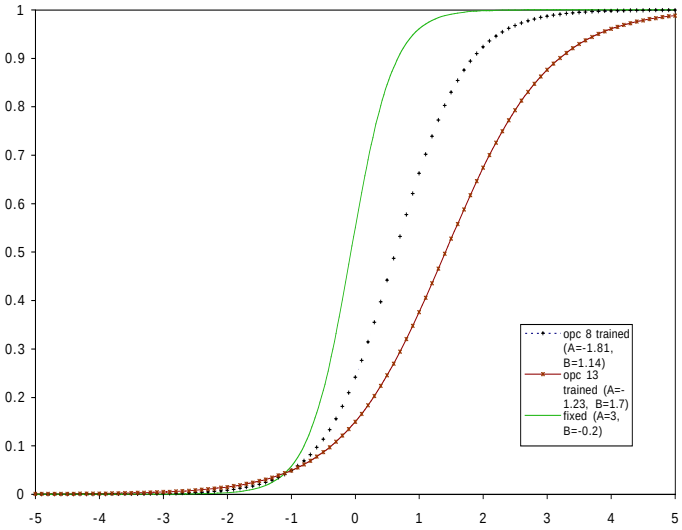



Figure 6: Sigmoid Fitting vs. Margin Boosting.

the SVM outputs such that $P(y = 1|f) \geq 0.95$ for $f \geq 1$, $P(y = 1|f) \leq 0.05$ for $f \leq -1$, and $P(y = 1|f) = 0.5$ for $f = 0$. This function will boost the confident outputs substantially.

Figure 6 plots two typical sigmoid fitting functions trained by the Platt's algorithm for two OPC classes, classes 8 and 13. The figure also shows our margin-boosting function (the left-most curve). Using Platt's method, while a low confidence SVM output (where $f \leq -1$) is fitted to the desired range $P(y = 1|f) \leq 0.05$, a highly confident output (where $f \geq 1$) remains in an ambiguous region (OPC-8 function gives $P(y = 1|f) \geq 0.24$ and OPC-13 gives $P(y = 1|f) \geq 0.38$). Using margin boosting, we map SVM margins that are greater than one to a probability of one and neutralize the negative effect of SVM margins between $-1$ and 1.

### 4.2 Error Reduction

To address the third shortcoming (classification variance), we use the error reduction procedure suggested in [7]. For each pairwise classifier separating class $i$ from $j$, an additional classifier is trained to separate $i$ and $j$ from the other classes. This produces a new SVM output $\hat{q}_{ij}$ to estimate the probability that point $\mathbf{x}$ belongs to class $i$ or $j$. The new $\hat{p}_i$ is then computed as follows:

$$\begin{pmatrix} +1 & -1 & 0 & 0 \\ +1 & 0 & -1 & 0 \\ +1 & 0 & 0 & -1 \\ 0 & +1 & -1 & 0 \\ 0 & +1 & 0 & -1 \\ 0 & 0 & +1 & -1 \end{pmatrix} \quad \begin{pmatrix} +1 & +1 & -1 & -1 \\ +1 & -1 & +1 & -1 \\ +1 & -1 & -1 & +1 \\ -1 & +1 & +1 & -1 \\ -1 & +1 & -1 & +1 \\ -1 & -1 & +1 & +1 \end{pmatrix} \tag{16}$$

PWC Matrix  Additional CC Matrix: $D$

Figure 7: Decomposition Matrices

$$\hat{p}_i = \frac{2}{K(K-1)} \sum_{j \neq i} \hat{p}_{ij} \cdot \hat{q}_{ij}. \tag{15}$$

Figure 7 shows the decomposition matrix $D$ for the additional classifiers, which we call correcting classifiers (CC), for four classes. Row 1 of the matrix is for classifier 1, which separates class 1 and 2 from the rest.

In Equation 15 we use CC (which estimates $\hat{q}_{ij}$) to augment PWC (which only estimates $\tilde{p}_{ij}$) and call the resulting scheme PWC_CC. Adding CC to PWC boosts the decision confidence of PWC. The increase in decision confidence results from the fact that CC boosts the *margins* of PWC. We follow Schapire's definition [17] to define margin $m_i$ as $(p_{i,y_i} - \max_{y \neq y_i} p_{i,y})$; that is, the probability (or the vote) of the true class minus the probability of the most popular wrong class. Our empirical study in Section 5 shows that CC indeed increases PWC's margins and significantly improves PWC's classification accuracy. This margin interpretation differs from an earlier theory [7, 19] that a voting scheme helps because the increased redundancy can correct classification errors.

Note that CC can itself be used as a voting scheme. Suppose we have $L$ correcting classifiers for $K$ classes and the normalized SVM output of each is $f_l(\mathbf{x})$, which is between $-1$ and 1. Together with the $L \times K$ decomposition matrix $D$, we classify a point $\mathbf{x}$ by:

$$\arg \max_k \sum_l (f_l(\mathbf{x}))\mathbf{D_{lk}}, \tag{17}$$

where $D_{lk}$ is an element in the decomposition matrix $D$ for classifier $l$.

# 5 Experiments

For our empirical evaluation of our learning methods we used a 1,920 fifteen-category image dataset collected from Corel Image CDs and the Internet. The 1,920 images in this dataset belong to fifteen categories — *architecture*, *bears*, *clouds*, *elephants*, *fabrics*, *fireworks*, *flowers*, *food*, *landscape*, *people*, *objectionable images*, *textures*, *tigers*, *tools*, and *waves*. In this set, a few categories were added to increase learning difficulty for SVMs. To make the landscape category more difficult to learn, we placed images of tigers and elephants on landscape and water backgrounds. The objectionable images can be confused with people wearing little clothing. We added colorful fabrics and food to interfere with flowers. Various texture images (e.g., skin, brick, grass, water, etc.) were added to raise the learning difficulty for all categories.

We have two major types of features [6], color and texture. For color, we have 12 color bins for the 11 culture colors (*black*, *white*, *red*, *yellow*, *green*, *blue*, *brown*, *purple*, *pink*, *orange*, and *gray*) and one outlier bin. We characterize each color with color histograms, color mean (in HSV channels), color variance (in HSV channels), elongation and spreadness. Texture consists of vertical, horizontal and diagonal orientations. For each orientation, we have texture energy mean, variance, elongation and spreadness. This gives us 12 texture features. For each of these, we extract coarse, medium and fine texture features. Altogether, we have 108 color and 36 texture features.

We divided our experiments into two main parts:

- In part one, we first evaluated our margin boosting scheme against the sigmoid fitting method, and then used the OPC scheme to evaluate SVM kernels and parameters settings.
- In part two, we experimented with various ensemble schemes for combinining binary classifiers to predict the class of an example. In particular, we compared our error reduction method with the ECOC error correction scheme. We employed margin boosting and used the optimal kernel and parameters setting found in part one.

| Scheme | Sigmod Fitting | Margin Boosting |
|---|---|---|
| OPC | 14.9 | 11.8 |
| PWC2 | 30.0 | 12.9 |
| CC | 13.35 | 11.5 |
| ECOC1 | 25.0 | 13.1 |
| ECOC2 | 24.1 | 12.3 |

Table 2: Error Rates (%) of Using Sigmoid vs. Margin Boosting Mapping.

To measure classification performance, we used classification error. We ran each experiment 10 times. At each run, we randomly selected a percentage of images from the dataset to be used as training examples for SVMs. The remaining images were used for classification. The results presented represent the average of 10 runs.

### 5.1 Part One Results

We first evaluated two methods for mapping SVM margin to probability: sigmoid fitting and margin boosting. We compared the effect of the mapping methods on the class prediction accuracy of OPC, PWC2, CC, ECOC1, and ECOC2. (We will discuss how ECOC1 and ECOC2 are constructed in Section 5.2.) Using 80% of images for training, Table 2 shows the average error rates of using sigmoid mapping versus that of using our margin boosting mapping. Our margin boosting improves classification accuracy drastically: PWC2, ECOC1, and ECOC2 by 100%, and OPC and CC by about 20%.

Employing the margin boosting mapping, we next evaluated SVMs using three kernels: polynomial, Gaussian and Laplacian. We used OPC to perform kernel evaluation. For each kernel, we varied kernel parameters such as degree of the polynomial and $\gamma$ for RBFs. Using the best kernel setting, we proceeded to examine the impact that different feature resolutions have on the performance. First, we looked at the difference between using only the color histograms and the full color features set. Next, we varied the resolution of texture from coarse to fine and compared the results for each of them.

Figure 8(a-c) shows the average error rate for each type of kernel with different per-

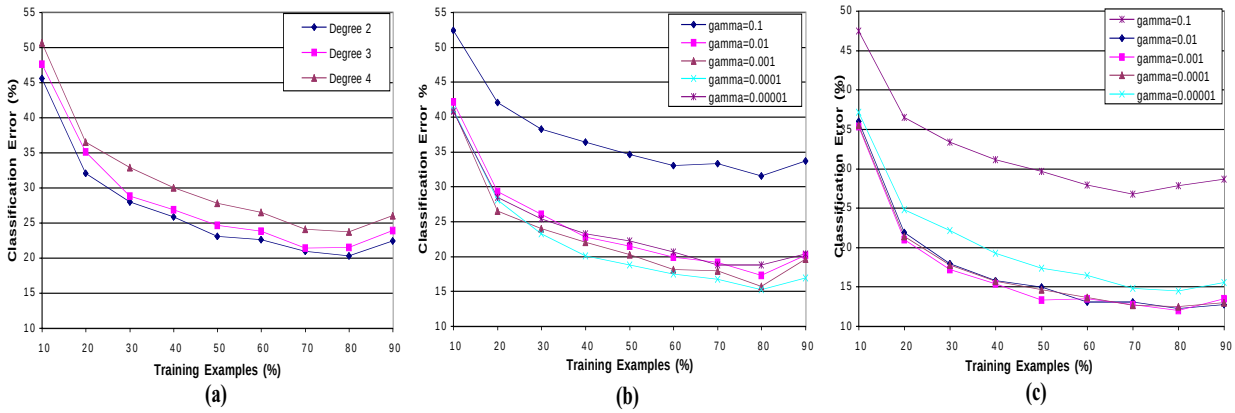



Figure 8: Classification Errors for Three Kernels (a) Polynomial (b) Gaussian (c) Laplacian.

centages of the dataset used as training examples. The kernels that gave the best results are polynomial with degree 2, Gaussian with $\gamma = 0.0001$ and Laplacian with $\gamma = 0.001$. Among the three kernels, the Laplacian kernel gave the lowest classification error of 12% using 80% of dataset of training examples. We used Laplacian kernel with $\gamma = 0.001$ for all subsequent experiments.

Table 3 shows the classification matrix using Laplacian kernel with $\gamma = 0.001$. The number shown is the sum of the 10 runs of learning and classification. We see that categories *bears*, *elephants* and *tigers* often get confused with each other and hence wrongly classified. We also see a similar trend in categories *clouds*, *landscape* and *waves*. The categories that perform best are *fireworks*, *food*, *objectionable images*, *texture* and *tools*.

## 5.2 Part Two Results

Again, in this part, we experimented with various ensemble schemes for combinining binary classifiers to predict the class of an example. Using margin boosting and the optimal kernel and parameters setting found in part one, we compared the error reduction method with the ECOC error correction scheme.

Figure 9(a) shows the results using different decision functions in Equation 7 with different percentages of training examples. We see that PWC1 and PWC4 (their lines are overlapping) give better results than PWC2 and PWC3. Both schemes are similar in that

| Categories | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 arch. | 290 | 1 | 0 | 4 | 3 | 8 | 0 | 4 | 14 | 1 | 7 | 0 | 0 | 0 | 1 |
| 1 bears | 0 | 162 | 2 | 6 | 1 | 0 | 4 | 0 | 14 | 1 | 2 | 0 | 5 | 2 | 10 |
| 2 clouds | 1 | 4 | 178 | 0 | 0 | 0 | 6 | 0 | 10 | 0 | 3 | 1 | 0 | 2 | 0 |
| 3 eleph. | 10 | 13 | 0 | 160 | 2 | 0 | 0 | 0 | 6 | 1 | 0 | 0 | 0 | 0 | 0 |
| 4 fabric | 7 | 0 | 1 | 6 | 169 | 0 | 2 | 1 | 5 | 0 | 1 | 5 | 0 | 0 | 0 |
| 5 firework | 0 | 0 | 0 | 0 | 0 | 172 | 0 | 4 | 0 | 1 | 3 | 0 | 3 | 0 | 0 |
| 6 flower | 0 | 1 | 2 | 1 | 1 | 0 | 366 | 6 | 1 | 3 | 10 | 0 | 0 | 0 | 0 |
| 7 food | 0 | 0 | 0 | 0 | 5 | 0 | 3 | 318 | 1 | 1 | 5 | 0 | 1 | 1 | 0 |
| 8 landsca. | 6 | 2 | 12 | 6 | 1 | 1 | 8 | 1 | 236 | 4 | 14 | 4 | 0 | 1 | 19 |
| 9 obj. img | 2 | 0 | 0 | 3 | 0 | 0 | 5 | 1 | 0 | 356 | 16 | 0 | 3 | 0 | 1 |
| 10 people | 6 | 7 | 1 | 3 | 2 | 1 | 15 | 6 | 9 | 13 | 233 | 0 | 8 | 2 | 0 |
| 11 text. | 0 | 0 | 0 | 1 | 3 | 0 | 0 | 0 | 5 | 0 | 0 | 187 | 0 | 0 | 0 |
| 12 tigers | 1 | 4 | 0 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 2 | 1 | 153 | 0 | 0 |
| 13 tools | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 213 | 0 |
| 14 waves | 1 | 2 | 6 | 0 | 0 | 0 | 0 | 0 | 15 | 0 | 0 | 1 | 1 | 0 | 168 |

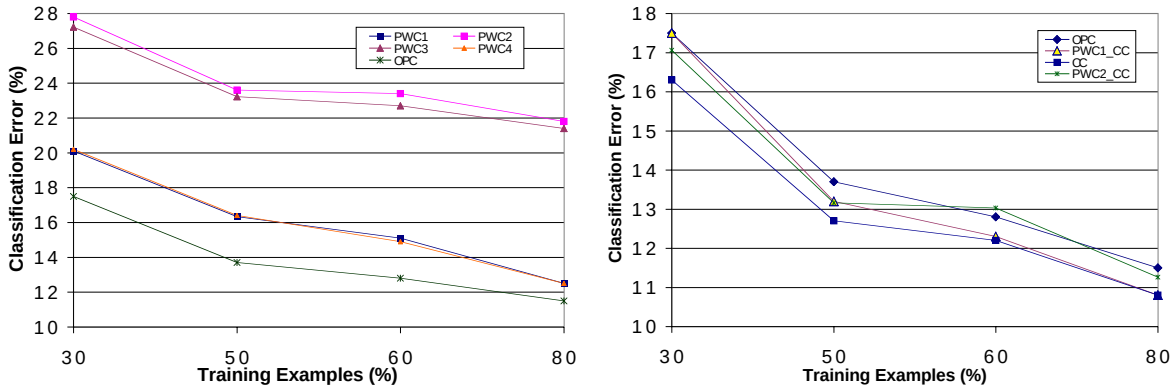
Table 3: Classification matrix using Laplacian kernel with $\gamma = 0.001$ and 80% training examples. Row 1 shows that 290 images in the architecture class are correctly classified, and the rest of the numbers constitute errors; 14 images are wrongly classified as landscape images.

positive SVM decision output is assigned more importance. Figure 9(a) also shows the results of the OPC. We observe that OPC, a simpler scheme, actually outperforms PWC1 and PWC4.

Next, we evaluated how an error reduction scheme like CC could improve the classification accuracy for the PWC schemes. From Figure 9(b), we see that CC improves the result of PWC1 (and PWC4) by 3% on average (see error curve for PWC1_CC) and improves the result of PWC2 (and PWC3) by 10% on average (see error curve for PWC2_CC). Using only CC for classification yields an equally good result. CC is a competitive scheme, performing as well as a combination of PWC and CC. Table 4 shows the error rates by various schemes. We see that PWC_CC and CC work best for most categories.

| Scheme | Arc. | Bears | Elep. | Clou. | Fab. | Fire. | Flo. | Food | Land. | Obj. | Peo. | Text. | Tig. | Tools | Wa. |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| OPC | **12.0** | 25.3 | 15.4 | 16.1 | **14.9** | 6.5 | 8.7 | 7.8 | 21.2 | 4.9 | 17.7 | **3.5** | 8.5 | **0.5** | 13.9 |
| PWC1 | **12.6** | **21.4** | 16.7 | 15.6 | 19.3 | 5.4 | 7.5 | **5.0** | 27.3 | 6.4 | 23.6 | **4.0** | 11.9 | **1.5** | 13.4 |
| CC | **12.3** | 25.2 | 13.0 | **14.2** | **15.5** | 5.9 | 7.2 | **5.3** | **19.7** | **4.4** | **15.8** | 6.6 | 9.1 | **0.5** | **12.3** |
| PWC_CC | **11.7** | **21.9** | **10.7** | **13.7** | **15.5** | **4.8** | **6.9** | **5.3** | 23.5 | **4.6** | 21.4 | **4.6** | **7.4** | **1.0** | **12.3** |

Table 4: Error Rates of Four Representative Schemes (using 80% of dataset as training examples).




(a) PWC Schemes vs. OPC

(b) OPC, PWC1_CC and CC

Figure 9: Classification Error vs. Classification Scheme

### 5.2.1 Why CC Helps

To understand why a correction classifier such as CC helps PWC, we compared the *margins* of training examples and testing data for PWC, PWC1_CC, and PWC2_CC. Again, we followed Schapire's definition [17] to define margin $m_i$ as $(p_{i,y_i} - \max_{y \neq y_i} p_{i,y})$; that is, the probability (or the vote) of the true class minus the probability of the most popular wrong class. A positive large margin can be interpreted as a "confident" correct classification.

| Scheme | Training | | Correctly Classified | | Misclassified | |
|---|---|---|---|---|---|---|
| | Average Margin | Std | Average Margin | Std | Average Margin | Std |
| PWC (PWC2) | 0.0020 | 0.0014 | 0.0017 | 0.0015 | −0.0007 | 0.0006 |
| PWC_CC (PWC2_CC) | 0.0038 | 0.0072 | 0.0040 | 0.0073 | −0.074 | 0.051 |
| PWC1_CC | 0.1843 | 0.1926 | 0.1775 | 0.1869 | −0.1586 | 0.1283 |

Table 5: Margins of Various Schemes.

Table 5 shows the margins of three schemes (note that PWC_CC is the same as PWC2_CC because PWC2 does not perturb PWC's results) collected from the training examples and from the testing data. The first column lists the average margins of training examples, the second column the average margins of the correctly classified testing data, and the third column the average margins of the misclassified testing data. In all three columns, the margins of PWC_CC are clearly much wider than the margins of PWC. We could therefore say that a classification decision made by PWC_CC has higher confidence than that

made by PWC. We are not surprised by the result that PWC_CC indeed achieves higher classification accuracy than PWC.

Now, let us compare the margins of PWC1_CC and PWC2_CC. They differ only in their decision functions (see Figure 2). PWC1_CC attempts to increase the margins by "artificially" polarizing positive and negative votes. Indeed, we can see in the table that the margins of PWC1_CC are much wider than the margins of PWC2_CC. However, this "artificial" boost of margins does not translate to better accuracy. (Figure 9(b) shows that PWC1_CC and PWC2_CC have almost the same accuracy.) Why?

As we discussed in Section 4, a classification decision made by a classifier with higher confidence is less likely to be perturbed by employing different decision functions (see Figure 4). This can be observed by comparing the classification errors of PWC1 and PWC2 before and after applying CC. Revisiting Figure 9, we see in 9(a) that the gap between PWC1 and PWC2 is about 10%, whereas in 9(b) PWC1_CC and PWC2_CC have about the same error. We believe that once CC boosts the margins of PWC_CC, any attempt by decision functions to perturb the results will be insignificant. In other words, only when the base classifier has little confidence in its decision will decision functions affect the results significantly. In summary, we believe that CC reduces classification variance by increasing margins (confidence) of the base classifier, and therefore, it helps achieve higher classification accuracy.

#### 5.2.2 Does Error Correction Help

For the ECOC ensemble scheme, we experimented with randomly-generated codes and those produced by the randomized hill-climbing algorithm (designed to maximize Hamming distance between pairs of codes). We noticed that the improved codewords produced by the algorithm did not yield better results. Table 6 shows the error rates and Hamming distance $d$ of random codewords and the improved codewords for two codelengths $L$. The Hamming distance of ECOC2 ($L = 210$) is more than double that of ECOC1 ($L = 105$) but the error rate is only 0.8% better. It appears that having more error-correction capability does not necessarily translate to a lower error rate.

| | ECOC1 ($L = 105$) | | ECOC2 ($L = 210$) | |
|---|---|---|---|---|
| | Random | Improved | Random | Improved |
| Error Rate (%) | 13.09 | 13.35 | 12.3 | 12.57 |
| Hamming Distance | 38 | 39 | 88 | 93 |

Table 6: Error rates and Hamming distance for randomly generated and improved coded-words used in ECOC scheme

| Schemes | OPC | CC | PWC2_CC | ECOC1 | ECOC2 |
|---|---|---|---|---|---|
| Error Rate (%) | 11.8 | 11.8 | 11.8 | 13.09 | 12.3 |
| Hamming Distance | 2 | 26 | 26 | 38 | 88 |

Table 7: Classification errors and Hamming distance for various voting schemes

Since the ECOC ensemble scheme has good error-correction capability, we compared OPC, CC, and PWC2_CC with the randomly-generated codes ECOC1 and ECOC2. Table 7 shows the classification error for various voting schemes using 80% of the dataset for training. It is observed that OPC, CC, and PWC2_CC schemes give comparable good performance of 11.8% error rate. From Table 7, we see that the Hamming distance for OPC is 2, and for both PWC2_CC and CC is 26. ECOC scheme performs worse than OPC, PWC2_CC, and CC even though it has a much larger Hamming distance.

## 6 Conclusions and Future Work

In this paper, we have shown that using Laplacian kernel, SVM-based ensemble schemes can achieve good accuracy for classifying images. We have also shown that by normalizing the decision outputs produced by SVMs and by employing error reduction schemes, the class prediction accuracy of OPC, PWC and ECOC can be enhanced. In addition, our empirical study showed that a scheme that has better error-correction capability does not necessarily perform better.

We are extending our classifier research in two directions. First, we are studying other alternative error reduction schemes (e.g., [7, 11, 16, 17]) and their effectiveness for improving the *confidence* of our SVM-classifier. Second, we will carefully study various methods'

contributions to the reduction in bias and variance of our SVM-based image classifier. (The precise role of the bias and variance of a classifier remains an open research issue [9, 10].)

## References


[1] E. L. Allwein, R. E. Schapire, and Y. Singer. Reducing multiclass to binary: A unifying approach for margin classifiers. *Journal of Machine Learning Research*, 1, 2000.

[2] L. Breiman. Bagging predicators. *Machine Learning*, pages 123–140, 1996.

[3] L. Breiman. Arcing classifiers. *The Annals of Statistics*, pages 801–849, 1997.

[4] C. Burges. A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2:121–167, 1998.

[5] E. Chang and B. Li. Mega — the maximizing expected generalization algorithm for learning complex query concepts (extended version). *Technical Report http://www-db.stanford.edu/∼echang/mega.ps*, November 2000.

[6] E. Chang, B. Li, and C. Li. Towards perception-based image retrieval. *IEEE Content-Based Access of Image and Video Libraries*, pages 101–105, June 2000.

[7] T. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. *Journal of Artifical Intelligence Research*, 2, 1995.

[8] J. Friedman. Another approach to polychotomous classification. *Stanford University Technical Report*, 1996.

[9] A. Grove and D. Schuurmans. Boosting in the limit: Maximizing the margin of learned ensembles. In *Proc. 15th National Conference on Artificial Intelligence (AAAI)*, 1998.

[10] G. James. Variance and bias for general loss functions. *USC Technical Report*, 2001.

[11] G. James and T. Hastie. *Error Coding and PaCT's*. 1997.

[12] T. Joachims. Transductive inference for text classification using support vector machines. In *Proceedings of the Sixteenth International Conference on Machine Learning*, pages 200–209. Morgan Kaufmann, 1999.

[13] B. Li, E. Chang, and C.-S. Li. Learning image query concepts via intelligent sampling. *Proceedings of IEEE Multimedia*, August 2001.

[14] M. Moreira and E. Mayoraz. Improving pairwise coupling classification with error correcting classifiers. *Proceedings of the Tenth European Conference on Machine Learning*, April 1998.

[15] J. Platt. Probabilistic outputs for svms and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*. MIT Press, 1999.

[16] J. Platt, N. Cristianini, and J. Shawe-Taylor. Large margin dags for multiclass classification. In *Advances in Neural Information Processing Systems 12*, pages 547–553. MIT Press, 2000.

[17] R. Schapire, Y. Freund, P. Bartlett, and W. Lee. Boosting the margin: A new explanation for the effectiveness of voting methods. In *Proceeding of the Fourteenth International Conference on Machine Learning*. Morgan Kaufmann, 1997.

[18] S. Tong and E. Chang. Support vector machine active learning for image retrieval. *Technical Report http://www-db.stanford.edu/∼echang/svmactive.ps*, March 2001.

[19] G. Valentini. Upper bounds on the training error of ecoc svm ensembles. *Technical Report*, Feb. 2001.

[20] V. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995.

[21] V. Vapnik. *Statistical Learning Theory*. Wiley, 1998.